

""

Cooper River Trading Co. –
Appraze Drive Folder Scanner

Scans a named Google Drive folder
(e.g. "Invoices" or "Inventory")
for new
image/PDF files, via the same
Apps Script backend used for
auth/storage.

Deliberately NOT a Gmail
integration – no OAuth app
registration, no
consent screen, no separate
credentials. The Apps Script runs
under your
own Google identity ("Execute as:
Me"), so it already has access to
your
own Drive. This is the tradeoff
you chose over the email-scanning

```
route.  
"""  
  
from dataclasses import dataclass  
from typing import List, Dict  
  
import requests  
import streamlit as st  
  
from auth import  
_apps_script_url, _token  
  
@dataclass  
class ScanResult:  
    success: bool  
    files: List[Dict] = None  
    error: str = ""  
  
def  
scan_invoice_folder(folder_name:  
str) -> ScanResult:  
    """Returns any not-yet-  
processed image/PDF files sitting  
in the named Drive folder."""
```

```

try:
    resp = requests.get(
        _apps_script_url(),
        params={"token":
_token(), "action":
"scan_folder", "folder_name":
folder_name},
        timeout=30, # Drive
+ base64 encoding of several
files can take a moment
    )
    resp.raise_for_status()
    data = resp.json()
    if not
data.get("success"):
        return
ScanResult(False,
error=data.get("error", "scan
failed"))
    return ScanResult(True,
files=data.get("files", []))
except Exception as e:
    return ScanResult(False,
error=f"connection error: {e}")

```

```

def
mark_files_processed(file_ids:
List[str], file_names: List[str])
-> bool:
    """Call after successfully
    extracting + adding a batch, so
    the same
    invoice never gets re-scanned
    and re-added on the next pass."""
    if not file_ids:
        return True
    try:
        resp = requests.get(
            _apps_script_url(),
            params={
                "token":
_token(),
                "action":
"mark_processed",
                "file_ids":
", ".join(file_ids),
                "file_names":
", ".join(file_names),
            },
            timeout=15,
        )

```

```
        resp.raise_for_status()
        return
    bool(resp.json().get("success"))
    except Exception:
        return False
```